

New York University Tandon School of Engineering
 Computer Science and Engineering

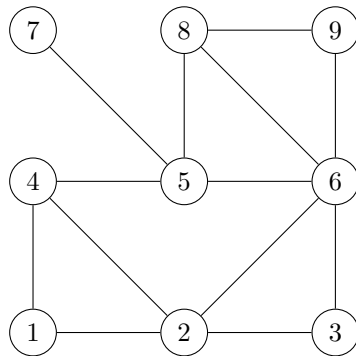
CS-GY 6033: Written Homework 4.
 Due Thursday, April 02, 2026, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

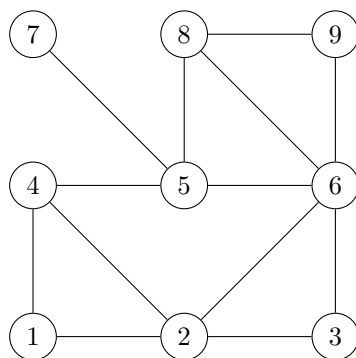
Problem 1

For the following problems, recall that (u, v) is a *tree edge* if node v is discovered while visiting node u during a breadth-first or depth-first search. Assume the convention that a node's neighbors are produced in ascending order by label. You do not need to show your work for this problem.

- Suppose a breadth-first search is performed on the graph below, starting at node 1. Mark every BFS tree edge with a bold arrow emanating from the predecessor.



- Suppose a depth-first search is performed on the graph below, starting at node 1. Mark every DFS tree edge with a bold arrow emanating from the predecessor.



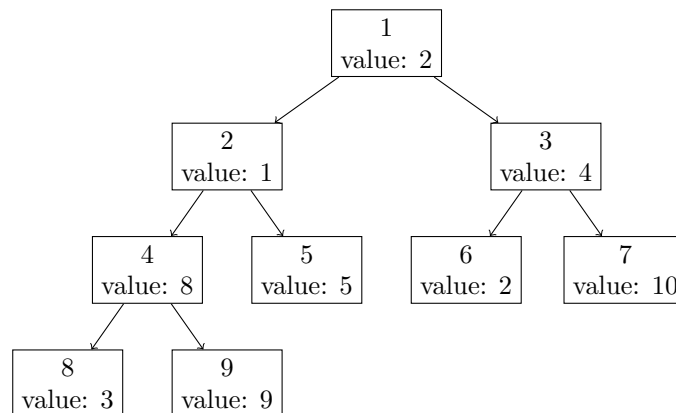
- Fill in the table below so that it contains the start and finish times of each node after a DFS is performed on the above graph using node 1 as the source. Begin your start times with 1.

Node	Start	Finish
1		
2		
3		
4		
5		
6		
7		
8		
9		

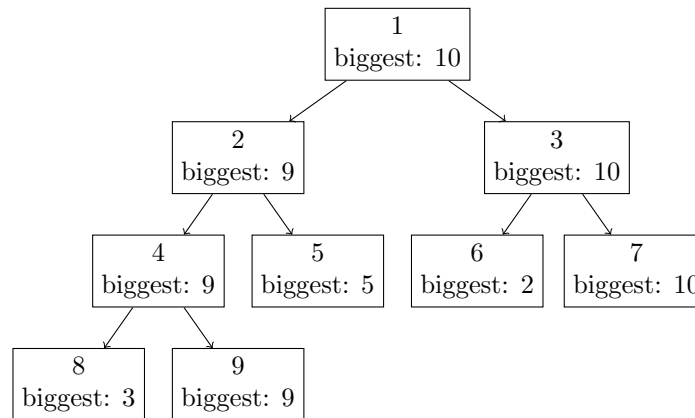
Problem 2

You are given a directed graph representing a tree and a dictionary `value` which contains a value for each node. Define the *biggest descendent value* of a node u to be the largest value of any node which is a descendent of u in the tree (for this problem, you should consider u to be a descendent of itself.)

For instance, given the following tree where each node's label is replaced by its value:



The *biggest descendent value* for each node is:



In a file named `biggest_descendent.py`, write a function `biggest_descendent(graph, root, value)` which accepts the graph, the label of the root node, and the dictionary of values and returns a dictionary mapping each node in the graph to its biggest descendent value.

The input is a graph represented as a Python dictionary (an adjacency list) and a dictionary `value`.

Problem 3

Suppose we are given a weighted, undirected graph $G = (V, E, \omega)$ in which the edge weights represent *similarity*; for example, the similarity between two users in a social network. Given a number λ , we will say that the *clusters* of G are the connected components of the graph after all edges whose weight is less than λ have been removed. There are other ways of defining the clusters of a weighted graph, but this is one natural way.

In `cluster.py`, write a function `cluster(graph, weights, level)` which computes the clusters of a weighted graph. Here, `graph` is a dictionary of lists representing an undirected graph, `weights(u, v)` is a dictionary mapping edges to their weights, and `level` is a number representing the level at which to find the clusters. Its return value should be a `frozenset`¹ containing `frozensets`; the inner `frozensets` should contain the nodes in a cluster.

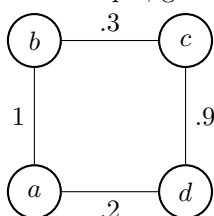
Your code should not modify the graph in any way and it should not create a copy of the graph. It should run in $\Theta(V + E)$ time.

Note that `weights` is a dictionary mapping edges to their weights: each key is a tuple (u, v) representing an edge. Since the graph is undirected, each edge appears *exactly once*, edges are stored in sorted order, i.e. (u, v) where $u < v$. To look up a weight, you must normalize the edge:

```

if u < v:
    w = weights[(u, v)]
else:
    w = weights[(v, u)]
    
```

For example, given the following graph:



```

graph = {
    "a": ["b", "d"],
    "b": ["a", "c"],
    
```

¹We use `frozensets` here because clusters are sets, but `sets` cannot be elements within other `sets` since they are not hashable.

```

    "c": ["b", "d"],
    "d": ["a", "c"]
}

weights = {
    ("a", "b"): 1,
    ("b", "c"): 0.3,
    ("c", "d"): 0.9,
    ("a", "d"): 0.2
}

```

The output when run with a level of 0.4 should be:

```

cluster(graph, weights, 0.4)
# frozenset([frozenset(['a', 'b']), frozenset(['c', 'd'])])

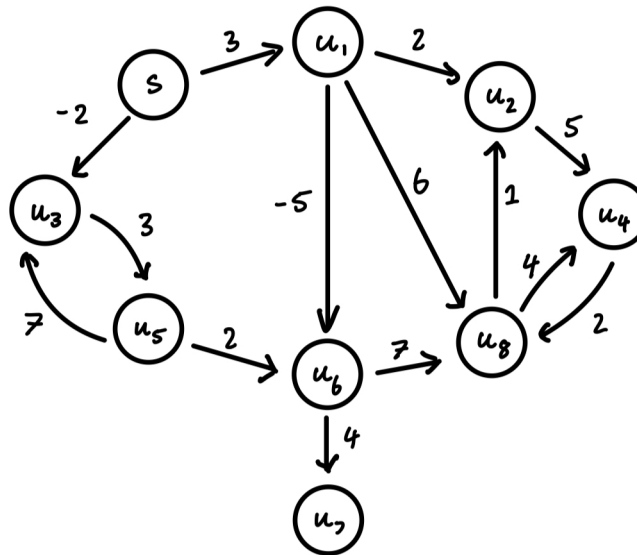
```

Note: you might see curly braces instead of square brackets in the output – that’s OK. Different versions of Python print frozensets differently.

Problem 4

Run Bellman-Ford on the following graph using node s as the source. Below each node u , write the shortest path length from s to u . Mark the predecessor of u by highlighting it or making a bold arrow. You can assume that `graph.edges` produces the graph’s edges in the following order:

$(u_3, u_5), (u_1, u_2), (u_5, u_6), (u_4, u_8), (u_6, u_7), (s, u_1), (u_5, u_3),$
 $(u_1, u_8), (u_6, u_8), (u_8, u_4), (s, u_3), (u_2, u_4), (u_1, u_6), (u_8, u_2)$

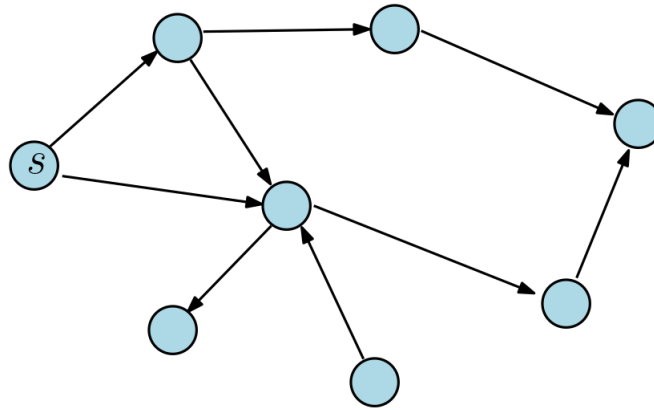


Problem 5

Recall that the Bellman-Ford algorithm (with early stopping) will terminate early if, after updating every edge, no predecessors have changed. Suppose it is known that the greatest shortest path distance in a graph $G = (V, E)$ has $\Theta(\sqrt{V})$ edges. What is the worst case time complexity of Bellman-Ford when run on this graph? State your answer using Θ notation.

Problem 6

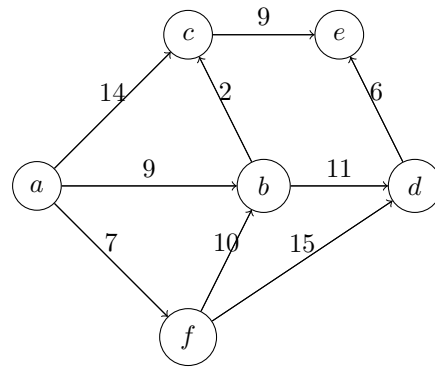
Suppose we are given a directed and weighted graph $G = (V, E)$ that look like the following figure. It is known that there are no negative cycles.



Suppose the Bellman-Ford algorithm **with early stopping** is run on this graph shown above using node s as the source (the edge weights are purposely not shown). In the worst case, how many iterations of the outer for-loop of Bellman-Ford will be performed? Briefly justify your answer.

Problem 7

Run Dijkstra's Algorithm on the following graph using node a as the source. Below each node u , write the shortest path length from a to u . Mark the predecessor of u by highlighting it or making a bold arrow.



Problem 8

True or False. Suppose $G = (V, E, \omega)$ is a weighted graph for which all edges are positive, except for those edges of a node s which may or may not be negative. If Dijkstra's algorithm is run on G with s as the source, the correct shortest paths will be found. Assume that the graph does not have any negative loops. Justify your answer.

Problem 9

Consider the following scenario:

Lucas is hungry, and he wants to find a restaurant to eat at on campus. Unfortunately, all the restaurants on campus have a wait time.

Lucas has a weighted undirected graph $G = (V, E, \omega)$ of the campus, where notable locations are nodes, and the edge weights denotes time it takes to traverse that edge. Among the notable locations, some are restaurants. Each restaurant is marked with an associated wait time, indicating how long Lucas would need to wait for food *after arriving at the restaurant*.

Design an algorithm that **for all possible locations** of Lucas, $\forall v \in V$, computes the minimum time Lucas needs to spend before he can get food. Explain your algorithm in words and state its time complexity.

To receive full credit, your solutions must have optimal time complexity.

You may assume the graph is implemented using dictionary of sets.

Problem 10

Modern AI accelerators (such as GPUs and multi-chip modules designed by NVIDIA) consist of thousands of compute units connected via high-speed on-chip interconnects (e.g., Network-on-Chip (NoC), NVLink-like fabrics). We can model such system as an undirected graph $G = (V, E)$ where:

- vertices represent compute units, memory controllers, or switches,
- edges represent communication links.

A node is **critical** if its removal increases the number of connected components of the graph. In other words, removing such a node disconnects parts of the system that were previously able to communicate.

This is undesirable in practice, as it indicates a *single point of failure*: the failure of one component (e.g., a router, switch, or controller) can isolate entire regions of the network or chip, leading to degraded performance or complete loss of functionality in those regions. A well-designed system should minimize or eliminate such critical nodes by introducing redundancy.

1. Design an algorithm to find all critical nodes in G .
2. Let $S \subset V$ be a set of high-priority nodes. We want to check whether there exists a node whose failure disconnects nodes in S from the rest of the graph. This is important because it identifies whether high-priority nodes depend on a single node to remain connected. If such a node exists, then a single failure can isolate important parts of the system, indicating poor reliability and the need for better redundancy.

Determine whether there exists a single node whose removal disconnects at least one node in S from the remaining nodes in $V \setminus S$. If yes, return all such nodes.